Seminario de Arquitecturas de Software:

Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Maní Molido 2, ahora más molido

Integrantes

#	Nombre completo
1	Adrián Xavier Brioso Jurado
2	Alejandro García Ruiz
3	Ana Laura Hernández Cutiño
4	Juan Carlos Yern Espinosa
5	Laura Alonso Rivero

Contribución individual por pregunta

Integrante	Pregunta(s) en las que trabajó
Adrián Xavier Brioso Jurado	5
Alejandro García Ruiz	4
Ana Laura Hernández Cutiño	3
Juan Carlos Yern Espinosa	6
Laura Alonso Rivero	1, 2

Repositorio público del proyecto

`https://github.com/Jcyern/Management-for-the-automatic-generation-of-delivery-routes/tree/Entregas-IS`

13 de noviembre de 2026

1. Análisis Comparativo de Comunicación y Acoplamiento

El escenario plantea una subasta en línea en la que, al finalizar, deben ejecutarse tres acciones: notificar al ganador, procesar el pago y actualizar el inventario. La comparación se realiza entre dos alternativas de microservicios —coreografía de eventos y orquestación mediante un mediador— y una Service-Based Architecture (SBA), en la que las tres acciones se ejecutan dentro de un mismo servicio de dominio de grano grueso.

1.1. Impacto en el acoplamiento dinámico

Para evaluar el acoplamiento es fundamental analizar el *connascence* dinámico, que examina cómo se comunican los componentes en tiempo de ejecución. Según Richards y Ford (2020), dentro de este tipo de acoplamiento destacan el *Connascence of Execution* (CoE), donde el orden de ejecución es crítico, y el *Connascence of Timing* (CoT), donde el tiempo de respuesta es determinante (cap. 3).

Microservicios con coreografía (topología broker). En este enfoque, el servicio de ventas emite un evento y los demás servicios (pago, inventario, notificación) reaccionan a él sin un coordinador central. El acoplamiento estático es mínimo, pues cada servicio solo conoce el contrato del evento, pero existe un alto acoplamiento dinámico implícito. Richards y Ford (2020) señalan que, aunque la coreografía preserva la filosofía altamente desacoplada, "problemas comunes como el manejo de errores y la coordinación se vuelven más complejos en entornos coreografiados" (cap. 17). Existe un fuerte *Connascence of Execution* porque el sistema depende de que el flujo de eventos ocurra en una secuencia lógica específica.

Microservicios con orquestación (topología mediator). Aquí se introduce un servicio mediador (por ejemplo, `OrderProcessor`). El acoplamiento dinámico aumenta porque el mediador conoce, invoca y espera la respuesta de los demás servicios. Como explican Richards y Ford (2020), el mediador impone un fuerte *Connascence of Timing*, pues "debe esperar hasta recibir el acuse de recibo de todos los tres procesos paralelos antes de pasar al siguiente paso" (cap. 14).

Service-Based Architecture (SBA). La SBA utiliza servicios de dominio de grano grueso (*coarse-grained*). El acoplamiento dinámico externo se reduce drásticamente porque el proceso completo de venta ocurre internamente en un solo servicio (por ejemplo, `OrderService`). Richards y Ford (2020) destacan que en la SBA ocurre una orquestación interna a nivel de clase en lugar de una orquestación externa de servicios distribuidos (cap. 13). Las llamadas de red se minimizan y se sustituyen por llamadas a métodos en memoria.

1.2. Implicaciones en tolerancia a fallos y consistencia de datos (ACID vs. BASE)

Microservicios (consistencia eventual / BASE). Debido a que en los microservicios cada servicio tiene su propia base de datos —principio derivado del *Bounded Context* del Domain-Driven Design—, no se pueden utilizar transacciones ACID tradicionales a través de las fronteras de los servicios; intentar hacerlo, por ejemplo con transacciones distribuidas

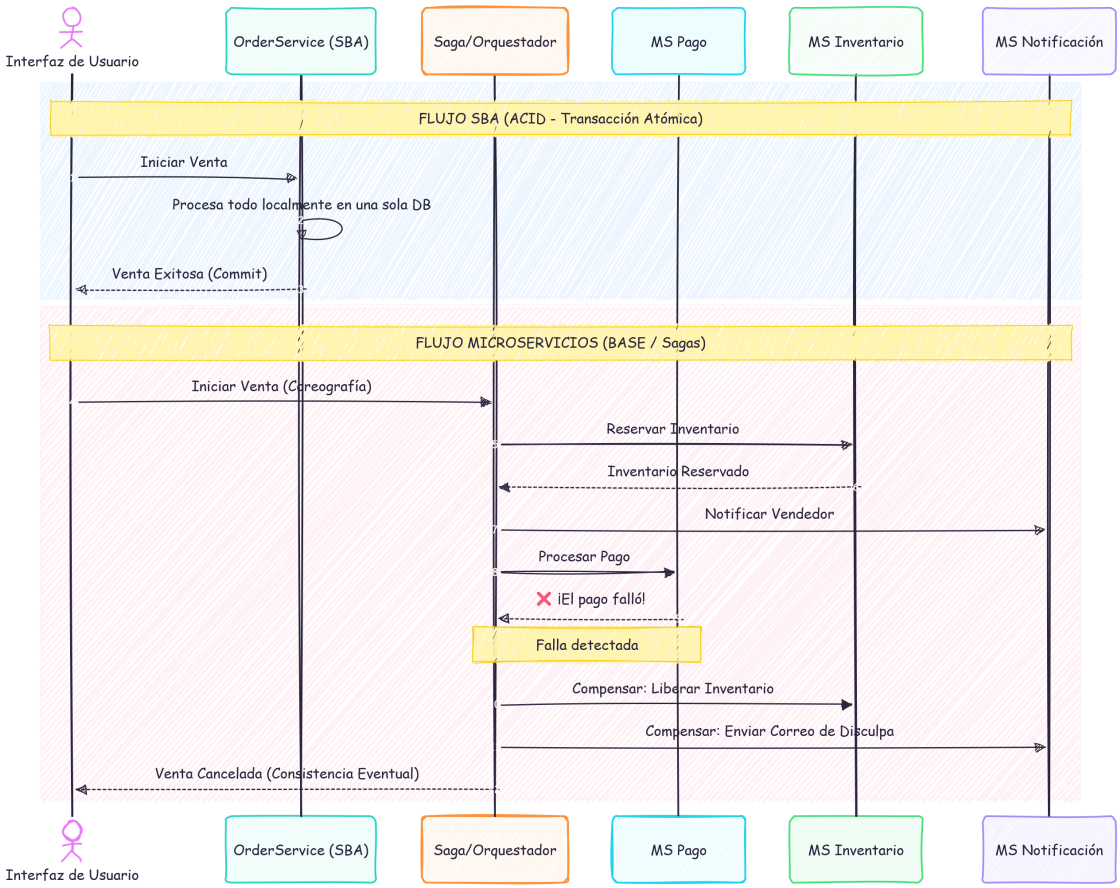


Figura 1: Diagrama de secuencia comparativo entre SBA (ACID) y microservicios con coreografía (BASE/Sagas).

2PC, es considerado un anti-patrón. En su lugar, se utilizan transacciones BASE (*Basic Availability, Soft state, Eventual consistency*) (Richards & Ford, 2020, caps. 13 y 17).

- **Tolerancia a fallos:** para resolver el estado inconsistente —vendedor notificado e inventario reservado, pero sin pago—, la arquitectura debe implementar el patrón Saga mediante transacciones compensatorias (Richards & Ford, 2020, cap. 17). Tal como explican Ford et al. en *Software Architecture: The Hard Parts* (2021), las sagas añaden una complejidad significativa al diseño, debido a que se debe implementar lógica explícita de "deshacer" (compensación). El sistema emitirá eventos para liberar el inventario y enviar un correo de disculpa al vendedor. El sistema no colapsa, pero la consistencia es eventual.

Service-Based Architecture (consistencia fuerte / ACID). Gracias a que el servicio es de grano grueso, las tres operaciones pueden ejecutarse dentro de una transacción ACID atómica contra una base de datos compartida (Richards & Ford, 2020, cap. 13). Esta es una ventaja clave de la SBA frente a los microservicios: mantiene la simplicidad transaccional de un monolito.

- **Tolerancia a fallos:** si el pago falla, el sistema simplemente ejecuta un *rollback*. "Todo lo agregado a la base de datos puede ser eliminado usando un rollback y se

envía un aviso al cliente de que el pago no se puede aplicar" (Richards & Ford, 2020, cap. 13). Los datos quedan íntegros y consistentes sin necesidad de lógica de compensación compleja.

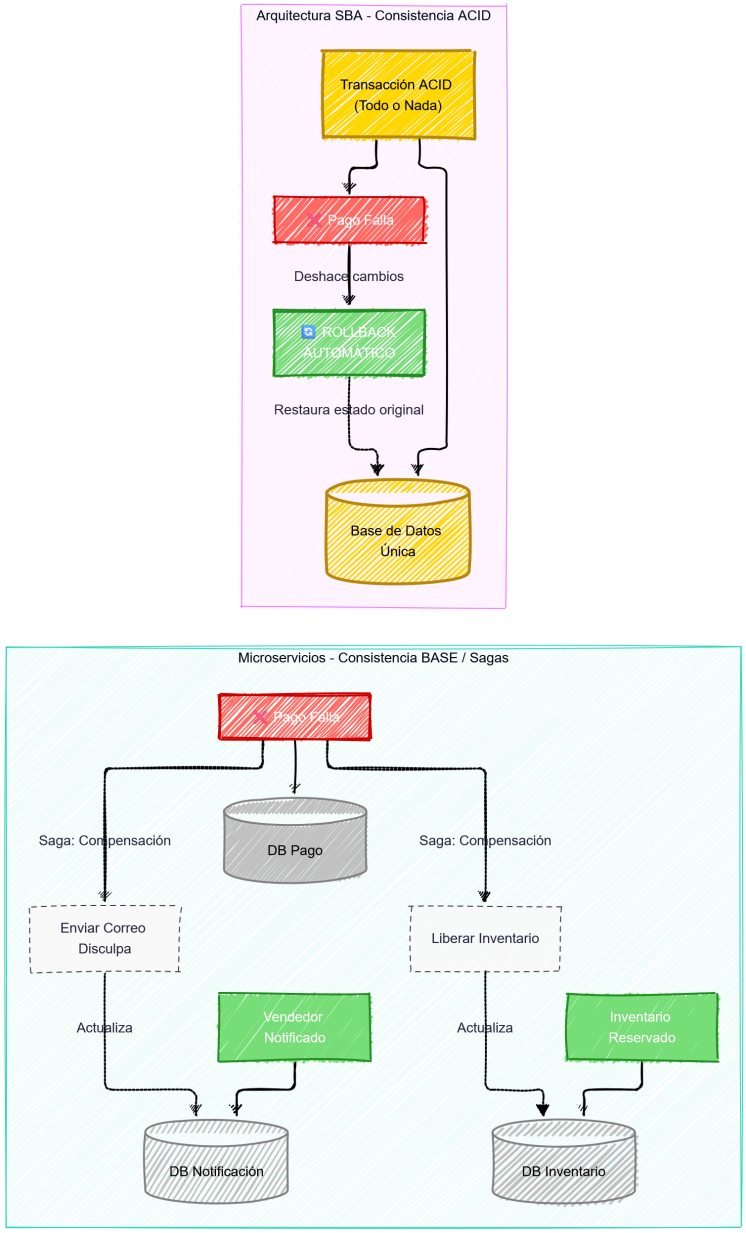


Figura 2: Comparación conceptual entre consistencia fuerte ACID (SBA) y consistencia eventual BASE (microservicios).

Aplicando la Primera Ley de la Arquitectura de Software ("todo es un trade-off") (Richards & Ford, 2020, cap. 1), la elección depende del objetivo del negocio:

- Si se prioriza la consistencia fuerte de datos y la simplicidad operativa, la SBA es superior, ya que evita el problema de las transacciones distribuidas y el costo de implementar sagas.

- Si se prioriza la escalabilidad extrema, el despliegue independiente y la agilidad, los microservicios —con sagas y coreografía u orquestación— son la mejor opción, aceptando el costo de la consistencia eventual y la complejidad del manejo de errores.

2. Diseño para Elasticidad Extrema y Consistencia de Datos

El escenario es el sistema de venta de entradas para conciertos descrito en el capítulo 15 de Richards y Ford (2020). Durante el lanzamiento de entradas para un artista popular, el sistema pasa de unos pocos usuarios a decenas de miles en segundos. En esta sección, para evitar la ambigüedad con la Service-Based Architecture de la pregunta anterior, se denomina Space-Based Architecture o arquitectura basada en espacio.

2.1. ¿Por qué la Space-Based Architecture es superior a los microservicios en este escenario?

Para entender por qué la arquitectura basada en espacio es la elección correcta, debe analizarse el cuello de botella de las arquitecturas tradicionales y de microservicios. Richards y Ford (2020) explican que, en las aplicaciones de alto volumen, el servidor de base de datos es el factor limitante final para las transacciones concurrentes. A medida que la carga aumenta, escalar los servidores web y de aplicaciones solo empuja el cuello de botella hacia la base de datos (cap. 15).

- **El problema con microservicios:** aunque ofrecen un gran desacoplamiento, cada servicio sigue dependiendo de su propia base de datos para persistir la información (*data isolation*). Si decenas de miles de usuarios intentan comprar entradas simultáneamente, los microservicios de inventario y pago colapsarán sus bases de datos debido a la latencia de entrada/salida del disco.
- **La ventaja de la arquitectura basada en espacio:** elimina la base de datos central como una restricción síncrona en el procesamiento transaccional. En su lugar, utiliza rejillas de datos replicadas en memoria (*in-memory data grids*) (cap. 15).
- **El motor de la elasticidad extrema:** la arquitectura basada en espacio califica con cinco estrellas en elasticidad y escalabilidad (cap. 15). Esto se debe al *Deployment Manager*, un componente del middleware virtualizado que monitorea continuamente los tiempos de respuesta y la carga de usuarios, y arranca o apaga dinámicamente unidades de procesamiento (*Processing Units*). Cuando las entradas salen a la venta, el sistema puede levantar cientos de instancias en segundos, incluso de forma proactiva antes del pico. Además, al procesar todo en memoria RAM, la latencia se reduce a milisegundos.

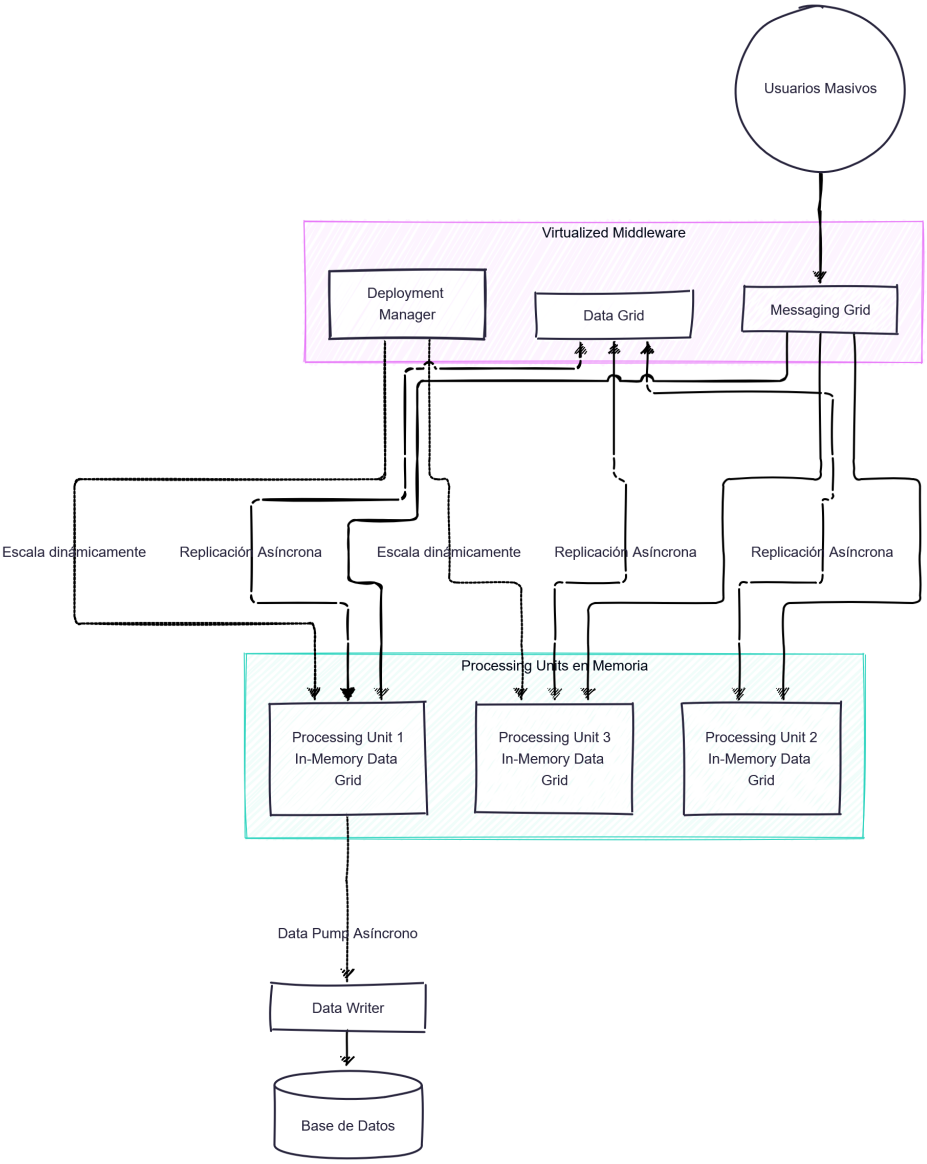


Figura 3: Topología general de una arquitectura basada en espacio (Space-Based Architecture).

2.2. Mitigación del riesgo de inconsistencia de datos en entradas numeradas

Se plantea un problema crítico: la arquitectura basada en espacio usa por defecto cachés replicadas. Si el usuario A compra el asiento 1, la actualización se replica a otras unidades de procesamiento de forma asíncrona (tarda menos de 100 ms) (Richards & Ford, 2020, cap. 15). En ese lapso, el usuario B podría comprar el mismo asiento 1 en otra unidad, provocando una condición de carrera (*double-booking*). Para mitigar este riesgo en un sistema donde la consistencia es crucial, el arquitecto debe cambiar la estrategia de almacenamiento en caché:

1. **Uso de caché distribuida para el inventario:** en lugar de caché replicada, se debe usar una caché distribuida centralizada para los datos de asientos. Richards y Ford (2020) explican que la caché distribuida admite altos niveles de consistencia de datos

porque los datos están todos en un solo lugar y no necesitan ser replicados (cap. 15).

2. **Bloqueos o transacciones atómicas:** el sistema debe usar mecanismos de bloqueo, como los que ofrecen productos de grid en memoria como Hazelcast o Apache Ignite (Richards & Ford, 2020, cap. 15), para asegurar que, durante la fracción de segundo en que un usuario selecciona un asiento, ese asiento quede bloqueado para los demás usuarios.

3. **Por qué no usar *near-cache*:** el libro advierte explícitamente que el modelo *near-cache* —que combina caché local en la unidad de procesamiento con caché distribuida— no se recomienda para esta arquitectura. Esto se debe a que las cachés frontales (locales) no se sincronizan entre sí entre diferentes unidades de procesamiento, lo que generaría inconsistencias de rendimiento y de datos entre los usuarios (cap. 15).

2.3. Mecanismo de trade-off: consistencia frente a rendimiento, tolerancia a fallos y complejidad operativa

La Primera Ley de la Arquitectura de Software indica que todo es un trade-off. Al elegir una caché distribuida en lugar de una replicada para garantizar la consistencia, el arquitecto y el CTO sacrifican rendimiento y tolerancia a fallos, y asumen una alta complejidad operativa. Richards y Ford (2020) proporcionan una tabla de decisión clara (tabla 15-1) donde la caché distribuida está optimizada para la consistencia, mientras que la replicada lo está para el rendimiento y la tolerancia a fallos (cap. 15).

Atributo	**Caché replicada**	**Caché distribuida**
Consistencia	Baja: las réplicas se actualizan de forma asíncrona	Alta: existe una única fuente de datos centralizada
Rendimiento	Alto: los datos se leen localmente en memoria	Menor: cada consulta viaja por la red al servidor central
Tolerancia a fallos	Alta: sobrevive si falla una réplica	Baja: el servidor central es un punto único de fallo
Complejidad operativa	Media	Alta: requiere coordinación de bloqueos, red y monitoreo
Uso recomendado	Datos estáticos o de referencia	Inventario dinámico y crítico, como los asientos

Cuadro 1: Comparación de estrategias de caché en una arquitectura basada en espacio, a partir de Richards y Ford (2020, cap. 15).

- **Sacrificio de la simplicidad y complejidad operativa:** al priorizar la consistencia sobre la agilidad, el CTO debe aceptar que la arquitectura basada en espacio es un estilo extremadamente complejo de probar y mantener (con una calificación de una estrella en simplicidad y testabilidad). Coordinar los bloqueos, las cachés y los *data pumps* requiere un equipo de operaciones altamente maduro. Si bien el despliegue de nuevas unidades es ágil gracias al *Deployment Manager*, simular y probar cargas extremas de cientos de miles de usuarios en producción es una tarea muy complicada y costosa (cap. 15).

- **Desventajas de la caché distribuida:** tiene un menor rendimiento que la replicada porque los datos se deben consultar por red a un servidor central, lo que agrega

latencia. Además, la tolerancia a fallos es baja: si el servidor de caché central se cae, todas las unidades de procesamiento quedan inoperativas, convirtiéndose en un punto único de fallo (*SPOF*) (cap. 15).

- **Estrategia de mitigación óptima (híbrida):** el arquitecto no debe elegir una sola opción. La recomendación de Richards y Ford es aprovechar las fortalezas de cada modelo (cap. 15):
  - Para el inventario de asientos, altamente dinámico y crítico, usar caché distribuida para garantizar que no se venda el mismo asiento dos veces (consistencia).
  - Para los datos del concierto, información estática o de referencia, usar caché replicada para un rendimiento ultrarrápido y alta tolerancia a fallos, evitando cuellos de botella innecesarios.

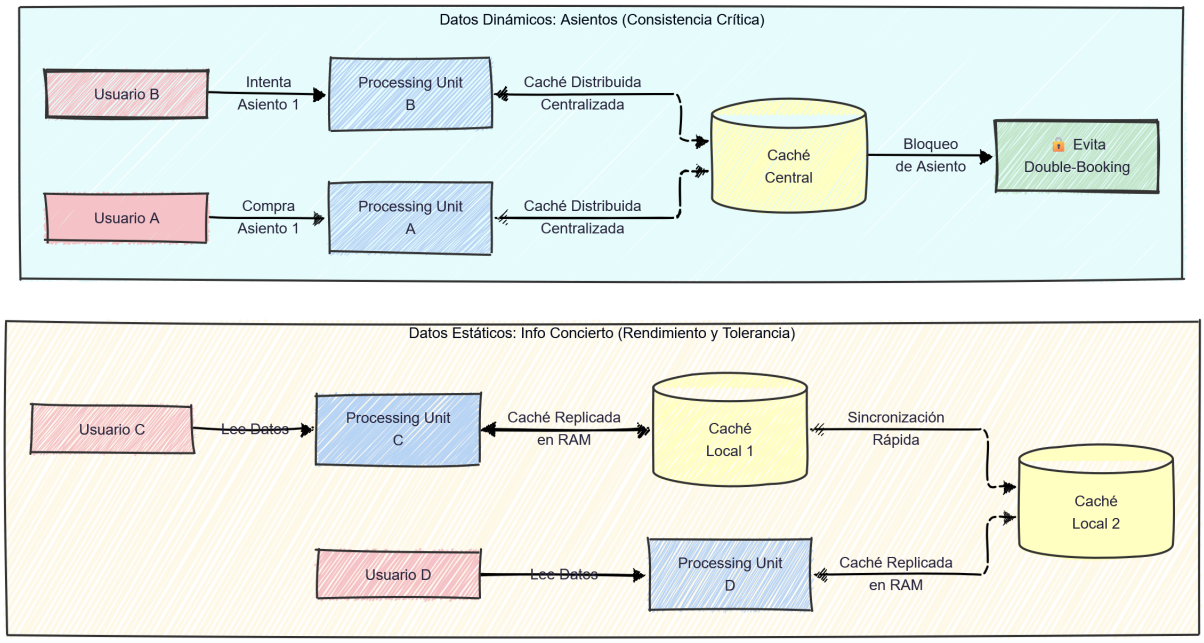


Figura 4: Estrategia híbrida de caché: distribuida para inventario dinámico y replicada para datos estáticos.

3. Análisis de Fallos y Resiliencia en Arquitecturas Dirigidas por Eventos

El escenario plantea una Event-Driven Architecture (EDA) con topología de broker en la que un *Event Processor* falla de forma repetida al procesar un tipo específico de mensaje debido a un error en los datos. La pregunta pide describir el impacto de ese fallo en el flujo y en la experiencia del usuario final, y proponer una solución arquitectónica basada en el *Workflow Event Pattern* del capítulo 14, detallando sus componentes y sus *trade-offs*.

3.1. Impacto del fallo en el flujo y en la experiencia del usuario

En una EDA con topología de broker no existe un mediador central que controle el flujo. El broker distribuye los eventos y cada *event processor* consume, procesa y publica nuevos eventos; los demás procesadores reaccionan a esos eventos sin que nadie tenga una visión global del estado de la transacción (Richards & Ford, 2020, pp. 241–247). Esta descentralización aporta desacoplamiento, escalabilidad, capacidad de respuesta y tolerancia a fallos, pero también elimina el control explícito del flujo: **no hay un componente que sepa en qué paso va una operación ni que pueda reintentar o compensar de forma coordinada** (Richards & Ford, 2020, pp. 246–247).

Si un *event processor* falla repetidamente al procesar un tipo concreto de mensaje por un error en los datos, lo que suele denominarse *poison message*, el impacto depende del modo de confirmación:

- **Con *auto-acknowledge***, el mensaje se elimina de la cola en cuanto se entrega. Si el procesador falla antes de completar el trabajo, el mensaje se pierde y nadie vuelve a intentarlo. El resto de procesadores continúa como si nada, pero la operación de negocio queda incompleta y en un estado inconsistente.
- **Con *client-acknowledge***, el mensaje permanece en la cola y se reentrega una y otra vez. El procesador entra en un bucle de reintento con el mismo mensaje defectuoso, lo que puede bloquear el consumo de los mensajes posteriores de esa cola, generar una acumulación (*backlog*) y producir efectos duplicados si el procesamiento no es idempotente.

En ambos casos, el problema es el mismo: **nadie es responsable de reparar el mensaje ni de reiniciar el flujo desde donde se quedó**. Richards y Ford (2020, p. 246) señalan expresamente que la topología de broker no ofrece control del *workflow*, ni gestión de errores, ni recuperabilidad, ni capacidad de reinicio. El sistema sigue procesando otros eventos, pero la transacción de negocio original queda atascada o perdida.

Desde la perspectiva del usuario final, el efecto es especialmente engañoso porque la comunicación es asíncrona. El usuario puede haber recibido un acuse de recibo y, sin embargo, la acción posterior nunca se completa. No recibe un error inmediato, no sabe qué paso falló y el soporte técnico no tiene un punto único donde consultar el estado. En un sistema de subastas, por ejemplo, el ganador podría no recibir la notificación, el pago no ejecutarse y el inventario no actualizarse, mientras el resto del sistema aparenta normalidad. La experiencia de usuario se degrada por silencio operativo: no hay error visible, pero el resultado prometido no llega.

3.2. Solución propuesta: Workflow Event Pattern

El *Workflow Event Pattern* es un patrón de arquitectura reactiva que aborda precisamente los problemas de gestión de errores en flujos asíncronos sin sacrificar la capacidad de respuesta (Richards & Ford, 2020, pp. 260–264). La idea central es separar el procesamiento normal del tratamiento del error: el procesador de eventos no se queda bloqueado intentando resolver el mensaje defectuoso; lo delega a un componente especializado y continúa con el siguiente mensaje.

El flujo sería el siguiente:

1. El *event producer* envía un mensaje a la cola.

2. El *event consumer* lo recibe y comienza a procesarlo.

3. Si se produce una excepción, por ejemplo un `NumberFormatException` porque el campo de unidades contiene el texto `"8756 SHARES"` en lugar de un número, el consumidor **no reintenta en el sitio ni se bloquea**. Inmediatamente delega el error a un **workflow processor** mediante un canal asíncrono, adjuntando el mensaje original y la información de la excepción.

4. El consumidor vuelve a leer el siguiente mensaje de la cola. De este modo, la capacidad de respuesta del sistema no se ve afectada: los mensajes posteriores siguen procesándose.

5. El **workflow processor** inspecciona el error e intenta reparar el mensaje de forma programática. En el ejemplo del libro, detecta que el problema es la palabra `SHARES` y la elimina, dejando `8756`; después reenvía el mensaje corregido a la cola original para que el consumidor lo procese de nuevo (Richards & Ford, 2020, pp. 262–263).

6. Si el workflow processor no puede reparar el mensaje porque el error es ambiguo, requiere una decisión de negocio o afecta a datos que el sistema no puede corregir automáticamente, lo envía a otra cola asociada a un **dashboard**. Este dashboard es una aplicación de consola o interfaz similar a un cliente de correo, atendida por una persona responsable, que revisa el mensaje, aplica una corrección manual y lo reenvía a la cola original, normalmente utilizando una cabecera `reply-to` (Richards & Ford, 2020, p. 261).

Los componentes adicionales, por tanto, son:

- **Workflow processor** o *workflow delegate*: componente que recibe los errores, intenta reparaciones automáticas y reencola los mensajes corregidos.
- **Dashboard**: interfaz para intervención humana cuando la reparación automática no es posible.
- **Cola o canal de errores**: canal asíncrono por el que el consumidor delega el error sin bloquearse.
- **Cola original**: destino al que se reenvía el mensaje reparado para su reprocesamiento.
- Opcionalmente, una **cola de mensajes no reparables** o *dead-letter queue* para auditoría y análisis.

Es importante subrayar que el workflow processor **no es un mediador central del flujo normal**. No orquesta la transacción ni conoce todos los pasos del negocio; solo actúa cuando hay un error. Por eso no reintroduce el acoplamiento ni el cuello de botella que caracterizan a la orquestación centralizada. Es un mecanismo de contención y reparación de errores, no un director de orquesta.

3.3. Cómo mitiga el problema y qué trade-offs introduce

El patrón mitiga el fallo en tres frentes:

- **Evita el bloqueo del consumidor**: al delegar el error y seguir leyendo, el procesador no se queda atascado en el mensaje defectuoso. La cola no se llena de mensajes no procesados por culpa de un único *poison message*.
- **Evita la pérdida silenciosa**: el mensaje no se descarta; se conserva, se repara y se reenvía, o se deriva a un humano. El sistema gana recuperabilidad sin necesidad de un mediador central.
- **Permite reparación automática o manual**: el workflow processor resuelve los errores deterministas, como quitar `SHARES` de un campo numérico, y escala los ambiguos al dashboard. Esto reduce la carga operativa y mejora la trazabilidad.

Ahora bien, el patrón introduce **trade-offs** que el arquitecto debe asumir y mitigar:

- **Procesamiento fuera de orden**. Cuando un mensaje se reenvía tras ser reparado, llega después de mensajes que originalmente iban detrás. En el ejemplo del libro, las órdenes de compra de una misma cuenta deben procesarse en orden: una venta de IBM debe ocurrir antes que una compra de AAPL dentro de la misma cuenta. Si el mensaje reparado se reenvía al final, se rompe ese orden (Richards & Ford, 2020, p. 264). La mitigación propuesta es almacenar temporalmente los mensajes posteriores de la misma cuenta en una cola auxiliar y liberarlos en orden FIFO una vez que el mensaje original ha sido reparado y procesado.
- **Mayor complejidad operativa**. Aparecen nuevos componentes como workflow processor, dashboard y colas de error que hay que desplegar, monitorizar y mantener. El sistema gana resiliencia, pero también superficie operativa.
- **Riesgo de duplicidad**. Si el mensaje se reenvía y el consumidor ya había producido algún efecto parcial antes de fallar, puede haber duplicados. Es necesario diseñar consumidores idempotentes y, cuando sea posible, usar claves de correlación o identificadores de mensaje para detectar reprocesamientos.
- **Latencia para el mensaje afectado**. El mensaje reparado no se procesa inmediatamente; pasa por el workflow processor y vuelve a la cola. Para el usuario final la operación afectada tarda más, aunque el resto del sistema sigue respondiendo.
- **Necesidad de observabilidad**. Sin un mediador central, es imprescindible instrumentar el flujo: métricas de errores por tipo de mensaje, tamaño de la cola de errores, tiempo medio de reparación, número de mensajes enviados al dashboard, etc. Sin esta visibilidad, el patrón puede ocultar problemas sistémicos.

Además, el patrón debe combinarse con mecanismos básicos de mensajería para cubrir los fallos que ocurren antes de que el consumidor pueda delegar el error. Richards y Ford (2020, pp. 265–267) recomiendan:

- **Colas persistentes** y **envío síncrono** para garantizar que el mensaje no se pierde entre el productor y la cola.

- ***Client acknowledge mode*** para que el mensaje no se elimine hasta que el consumidor lo haya procesado o delegado correctamente.
- **Transacciones ACID** en la persistencia final y ***last participant support*** **(LPS)** para retirar el mensaje de la cola solo cuando el dato ha quedado persistido.

3.4. Resultados

En una EDA con topología de broker, un *event processor* que falla repetidamente ante un mensaje defectuoso expone la debilidad estructural del estilo: **no hay un dueño del flujo que detecte, repare y reinicie la transacción**. El mensaje puede perderse o bloquear la cola, y el usuario final puede quedarse sin saber que su operación nunca se completó.

El *Workflow Event Pattern* responde a ese problema sin convertir el broker en un mediador central. El consumidor delega el error, sigue procesando y un **workflow processor** intenta reparar y reencolar; si no puede, un **dashboard** permite la intervención humana. El patrón mejora la resiliencia y la capacidad de respuesta, pero introduce procesamiento fuera de orden, complejidad operativa y riesgos de duplicidad que deben gestionarse con colas auxiliares por clave de negocio, idempotencia y observabilidad. La lección arquitectónica es que **la ausencia de un mediador central no elimina la necesidad de gestionar errores**, solo desplaza la responsabilidad hacia mecanismos especializados que deben diseñarse explícitamente.

4. El Dilema de la Granularidad y el Acoplamiento en Microservicios

Un equipo de desarrollo está migrando un sistema monolítico de gestión de recursos humanos a microservicios. Inicialmente crea servicios muy pequeños para cada entidad: `Empleado`, `Departamento`, `Nómina` y `Beneficios`. Pronto descubre que completar una operación simple, como contratar a un nuevo empleado, requiere una compleja coreografía de llamadas síncronas entre casi todos los servicios, lo que produce un alto acoplamiento y una fragilidad similar a la del monolito original.

4.1. El error fundamental en el diseño de los servicios

El error no fue crear servicios pequeños, sino descomponer por entidad o por tabla en lugar de hacerlo por contexto delimitado y capacidad de negocio (Richards & Ford, 2020, cap. 17; Ford et al., 2021, caps. 4 y 7). Servicios como `Empleado`, `Departamento`, `Nómina` y `Beneficios` parecen naturales desde el modelo de datos, pero no desde el negocio. Cada uno termina siendo un servicio CRUD anémico que expone operaciones sobre una entidad, mientras el proceso real de contratación queda fragmentado entre todos ellos. La consecuencia es un monolito distribuido: la operación "contratar a un empleado" requiere una cadena de llamadas síncronas entre casi todos los servicios, lo que produce acoplamiento temporal, latencia acumulada, propagación de fallos, fragilidad ante caídas parciales y una transacción de negocio repartida entre servicios que no pueden garantizar atomicidad conjunta. El diseño no redujo el acoplamiento del monolito original; solo lo trasladó a la red.

4.2. Por qué "Empleado" no es un concepto único y compartido

El problema de fondo es haber asumido que "Empleado" es un concepto único y compartido. En realidad, Empleado significa cosas distintas en contextos distintos:

- En **contratación**, es un agregado con datos contractuales, posición, departamento, fecha de alta y estado inicial.
- En **nómina**, es un sujeto fiscal con salario, deducciones, cuenta bancaria y periodicidad.
- En **beneficios**, es un miembro elegible con dependientes y plan de cobertura.

Intentar compartir un único modelo de Empleado entre todos esos contextos obliga a coordinarlos y los acopla por datos. El *bounded context* exige un lenguaje ubicuo propio y un modelo propio dentro de cada frontera, no un modelo único distribuido entre servicios (Richards & Ford, 2020, cap. 17).

4.3. Granularidad técnica frente a granularidad de negocio

Las guías de granularidad de servicios de Ford et al. (2021, caps. 4 y 7) y la descomposición por contextos delimitados de Richards y Ford (2020, cap. 17) coinciden en que el error es haber confundido granularidad técnica con granularidad de negocio. Un microservicio debe ser lo suficientemente pequeño para evolucionar de forma independiente, pero lo suficientemente grande para encapsular una capacidad de negocio, sus invariantes y su transacción local. Al bajar la granularidad al nivel de entidad, aumentan las llamadas remotas, la coordinación, el acoplamiento y la fragilidad. La guía práctica no es "un servicio por entidad", sino "un servicio por contexto delimitado y por agregado transaccional".

4.4. Nueva estrategia de descomposición: propósito, transacciones y coreografía

La nueva descomposición debe aplicar tres criterios articulados:

- **Propósito:** cada servicio debe corresponder a una capacidad de negocio cohesiva y tener una razón clara para cambiar. "Contratación" o "Gestión del Ciclo de Vida del Empleado" es un propósito; "Empleado" como entidad no lo es.
- **Transacciones:** cada servicio debe poseer los datos e invariantes que necesitan consistencia fuerte local. La creación del empleado, su contrato y su asignación inicial deben ocurrir dentro de una misma transacción local en el contexto de contratación. No tiene sentido repartir esa atomicidad entre cuatro servicios e intentar reconstruirla con llamadas síncronas.
- **Coreografía:** las interacciones entre contextos deben basarse en eventos de dominio y no en cadenas de llamadas síncronas. La nómina y los beneficios deben reaccionar a que un empleado fue contratado, no ser invocados obligatoriamente dentro de la operación de contratación (Ford et al., 2021, caps. 11 y 12).

Además, deben respetarse estos límites:

- Cada contexto es dueño de sus datos; no se comparte una entidad Empleado única entre contextos.
- Las fronteras rodean agregados transaccionales: lo que debe validarse junto y permanecer consistente junto vive en el mismo servicio.
- Las dependencias entre contextos se modelan como eventos, no como llamadas síncronas obligatorias.
- Las lecturas entre contextos se resuelven con proyecciones locales, no con consultas síncronas en cada operación.
- Los procesos largos se coordinan con un *process manager* o saga, no con una cadena de llamadas síncronas (Ford et al., 2021, caps. 9, 11 y 12).

4.5. Ejemplo de agrupación para la operación de contratación

La nueva descomposición podría quedar así:

Contexto	**Responsabilidad**	**Agregado y datos**	**Eventos**
Contratación y Ciclo de Vida del Empleado	Posee empleado, contrato y asignación inicial. Expone `contratarEmpleado` en una transacción local; no llama a Nómina ni a Beneficios.	`Empleado`, `Contrato`, `Asignación`	`EmpleadoContratado`
Organización	Posee los departamentos. Contratación mantiene una proyección local para validar sin llamada síncrona.	`Departamento`	`DepartamentoCreado`, `DepartamentoActualizado`
Nómina	Posee los registros de nómina. Se suscribe a `EmpleadoContratado` y crea el perfil de forma asíncrona e idempotente; reintenta sin bloquear.	`PerfilNomina`, `Deducción`, `CuentaBancaria`	`NominaCreada`
Beneficios	Posee elegibilidad e inscripción. Se suscribe a `EmpleadoContratado` e inicia beneficios de forma asíncrona; reintenta sin bloquear.	`Elegibilidad`, `Inscripción`, `Dependiente`	`BeneficiosIniciados`
Process Manager de Onboarding (opcional)	Escucha los eventos, mantiene el estado de onboarding y emite `OnboardingCompletado` o alertas.	`EstadoOnboarding`	`OnboardingCompletado`

Cuadro 2: Contextos delimitados propuestos para la operación de contratación, a partir de Richards y Ford (2020, cap. 17) y Ford et al. (2021, caps. 4 y 7).

El *process manager* de onboarding no reemplaza la coreografía con una cadena síncrona: observa eventos y coordina un proceso largo sin acoplar temporalmente a los servicios. Si se necesita seguimiento explícito, escucha eventos como `EmpleadoContratado`, `NominaCreada` y `BeneficiosIniciados`, y emite `OnboardingCompletado` o genera alertas cuando nómina o beneficios no hayan confirmado.

4.6. Consecuencias del rediseño

Con este diseño, la operación de contratación tiene una sola llamada síncrona principal al contexto de Contratación. Los efectos secundarios se propagan por eventos. Si Nómina falla, la contratación no se revierte necesariamente; el evento se reintenta, se envía a una cola de errores o el *process manager* marca el onboarding como pendiente. Si el negocio

exige que el empleado no quede activo hasta que nómina esté lista, se modela con un estado `PENDIENTE` y el *process manager* confirma la activación cuando recibe el evento de nómina. Eso sigue siendo consistencia eventual y coreografía, no una transacción distribuida.

La consistencia fuerte queda acotada al agregado de contratación, mientras que la consistencia entre contextos se asume explícitamente como eventual. El acoplamiento no desaparece, pero se transforma: se elimina el acoplamiento temporal de la cadena síncrona y el acoplamiento derivado de compartir un modelo único de Empleado, y en su lugar aparece un acoplamiento más débil a contratos de eventos que debe gestionarse con versionado, esquemas claros, idempotencia, reintentos y monitoreo (Ford et al., 2021, caps. 11 y 12). También aparecen datos duplicados y proyecciones locales, un costo que se acepta para ganar autonomía y reducir fragilidad. La clave está en que cada servicio sea dueño de sus datos y de sus decisiones, y en que la contratación no dependa de que Nómina y Beneficios estén disponibles en el mismo instante; esa es precisamente la diferencia entre descomponer por entidades y descomponer por contextos delimitados, propósitos de negocio, límites transaccionales y coreografía asíncrona.

5. Comparación de Atributos de Calidad y Trade-offs

Para un sistema de procesamiento de transacciones financieras, la decisión no puede basarse en qué estilo resulta más moderno, sino en qué requisito es una restricción dura y cuál es un objetivo deseable. Los tres requisitos del CTO no pesan igual: la consistencia absoluta es una restricción de corrección, auditoría y regulación; el despliegue independiente y la experiencia del equipo son objetivos organizativos. Esa jerarquía es la que inclina la balanza hacia la Service-Based Architecture y la que obliga a aceptar renuncias explícitas si se prioriza la consistencia.

5.1. Consistencia de datos absoluta (ACID)

El primer requisito es el que decide la comparación. La Service-Based Architecture, descrita por Richards y Ford (2020, cap. 13), se organiza en un número reducido de servicios de dominio de grano grueso, normalmente entre cuatro y doce, desplegables de forma independiente y apoyados en una base de datos compartida o parcialmente compartida. Esa base de datos común es precisamente lo que permite que una operación financiera crítica se ejecute como una transacción local ACID: un único motor relacional aplica atomicidad, consistencia, aislamiento y durabilidad sobre cuentas, saldos, movimientos y asientos contables, con restricciones de integridad, claves foráneas y bloqueos gestionados por el propio gestor. La consistencia fuerte no se simula ni se reconstruye entre servicios; existe porque hay una única fuente de verdad y porque la frontera transaccional coincide con la frontera del servicio que posee los datos. La consecuencia es que la SBA puede garantizar ACID de forma nativa para el núcleo del negocio, a costa de acoplar los servicios a un esquema de datos común.

La Microservices Architecture, en cambio, establece que cada servicio sea dueño de su propia base de datos (Richards & Ford, 2020, cap. 17), lo que es la base de su autonomía y de su despliegue independiente, pero también la razón por la que no puede ofrecer consistencia absoluta entre servicios. ACID solo se conserva dentro de cada servicio. Una transacción que afecte cuentas, límites, saldos y contabilidad distribuidos entre varios

servicios no puede ser atómica sin recurrir a una transacción distribuida, como el protocolo de confirmación en dos fases, que introduce coordinación bloqueante, latencia, contención de bloqueos, un coordinador que puede fallar y una disponibilidad peor (Ford et al., 2021, cap. 9). Esa alternativa contradice las premisas del estilo: autonomía, disponibilidad y evolución independiente. Por eso la respuesta habitual de los microservicios es la saga con compensaciones y consistencia eventual, junto con patrones como *outbox*, idempotencia, reintentos y reconciliación (Ford et al., 2021, caps. 11 y 12). Eso es BASE, no ACID: una compensación no equivale a un *rollback*, porque el dinero ya pudo moverse, notificarse o registrarse, y la corrección posterior no borra el efecto intermedio. En un sistema financiero, donde la exactitud del saldo y la auditabilidad no son negociables, esa diferencia es decisiva. Por tanto, para el requisito de consistencia de datos absoluta, la Service-Based Architecture es claramente más adecuada que la Microservices Architecture; los microservicios solo serían viables si el requisito se rebajara a consistencia fuerte dentro de cada contexto delimitado y consistencia eventual entre contextos, lo que ya es otra decisión de negocio.

5.2. Despliegue independiente de nuevas características

El segundo requisito se ve afectado de forma distinta en cada estilo. La SBA sí permite desplegar cada servicio por separado, pero lo hace con una granularidad gruesa: un cambio de negocio suele implicar desplegar un servicio completo y, si toca el esquema compartido, coordinarlo con los demás servicios que usan ese esquema. La base de datos compartida se convierte en un punto de acoplamiento de despliegue y en un contrato entre equipos; por eso las *releases* tienden a ser menos frecuentes, más grandes y coordinadas, y el tiempo de puesta en producción de una característica transversal aumenta. A cambio, el sistema tiene menos piezas móviles, menos *pipelines* y un modelo operativo más simple.

En los microservicios, el despliegue independiente es máximo en teoría (Richards & Ford, 2020, cap. 17): cada servicio tiene su repositorio, su *pipeline*, su base de datos y un radio de impacto pequeño, y puede liberar con frecuencia sin coordinar el esquema con los demás. Pero esa independencia es condicional: exige una plataforma madura de integración y entrega continuas, pruebas de contrato, versionado de APIs y eventos, observabilidad distribuida, descubrimiento de servicios y equipos con capacidad de operar lo que despliegan. Sin esa madurez, el resultado no es independencia, sino un monolito distribuido con despliegues igualmente coordinados y mucho más frágiles. Además, una característica de negocio suele atravesar varios servicios, de modo que la independencia técnica no elimina la coordinación entre equipos. En un dominio financiero, con controles de cambio, auditoría y validación regulatoria, la frecuencia de despliegue está limitada por el proceso de certificación más que por el estilo arquitectónico.

5.3. Equipo con experiencia mixta

El tercer requisito también favorece a la SBA. Menos servicios, una sola base de datos, llamadas síncronas y un modelo mental más cercano a la aplicación empresarial tradicional reducen la curva de aprendizaje y permiten que los perfiles menos experimentados sean productivos antes. Los perfiles *senior* pueden concentrarse en los límites del dominio, la integridad de los datos y la gobernanza del esquema. Los microservicios exigen competencias de sistemas distribuidos que un equipo mixto no necesariamente tiene: consistencia eventual, mensajería, sagas, idempotencia, trazabilidad distribuida, contenedores, orquestación,

infraestructura como código y operación de servicios. Sin una plataforma y una capacitación deliberadas, esa brecha se traduce en incidentes, deuda operativa y un monolito distribuido. La Microservices Architecture no es solo un estilo de código; es un modelo organizativo y operativo, y el equipo es parte de la decisión arquitectónica (Richards & Ford, 2020, cap. 17).

5.4. Trade-offs de priorizar la consistencia

Si el CTO prioriza la consistencia sobre la agilidad de despliegue, debe aceptar los siguientes trade-offs:

- **Despliegue más grueso y coordinado:** las características que tocan datos compartidos se despliegan juntas o en ventanas coordinadas. Se gana corrección, se pierde frecuencia y autonomía de *release*.
- **Acoplamiento por datos:** el esquema compartido es un contrato permanente. Exige propiedad clara de tablas, migraciones de expandir-contraer, compatibilidad hacia atrás, pruebas de integración y revisión de cambios de base de datos (Ford et al., 2021, cap. 9).
- **Menor autonomía de los equipos:** varios equipos dependen del mismo esquema y del mismo servicio de dominio, por lo que se necesitan gobernanza de datos, comités de cambio y un tren de *releases* común.
- **Menor elasticidad selectiva:** se escala el servicio completo o la base de datos, no una operación aislada. Las réplicas de lectura, el particionamiento y el almacenamiento en caché alivian la carga, pero no eliminan el cuello de botella de escritura del núcleo transaccional.
- **Menor heterogeneidad tecnológica:** un esquema y un motor de base de datos compartidos limitan la libertad de elegir tecnologías distintas por servicio.
- **Mayor radio de impacto:** un fallo o un cambio defectuoso en el servicio o en la base de datos compartida afecta a más funcionalidades. Se compensa con alta disponibilidad, copias de seguridad y recuperación a un punto en el tiempo, despliegues canario y observabilidad.
- **Menor velocidad para cambios transversales:** si el negocio exige más agilidad, habrá que relajar la consistencia en algún subconjunto o invertir en una plataforma que reduzca el costo de coordinación.

5.5. Estrategia híbrida y síntesis

La lectura correcta no es elegir un estilo puro, sino ubicar cada requisito donde de verdad aplica. El núcleo transaccional, es decir, cuentas, saldos, límites, movimientos y libro mayor, debe vivir en uno o pocos servicios de grano grueso con una base de datos relacional y transacciones locales ACID; ese es el sistema de registro y allí la SBA es la elección correcta. Alrededor de ese núcleo pueden convivir servicios independientes para notificaciones, reportería, análisis, detección de fraude, estados de cuenta o incorporación de clientes,

que consuman eventos mediante *outbox* o captura de cambios y que toleren consistencia eventual (Ford et al., 2021, cap. 11). Esos servicios satélite sí pueden desplegarse con la agilidad de los microservicios porque sus fallos no comprometen la exactitud del dinero. Si la organización insiste en microservicios para todo, la única forma de preservar la corrección es mantener el libro mayor como autoridad transaccional única, usar sagas solo para procesos que admitan compensación y no para el registro de movimientos, y añadir idempotencia, *outbox*, reconciliación y auditoría. Aun así, no se obtiene ACID absoluto entre servicios, sino consistencia fuerte local y consistencia eventual en los bordes.

En síntesis, para los tres requisitos planteados la Service-Based Architecture es la opción más adecuada: cumple la consistencia absoluta de forma nativa, ofrece despliegue independiente aunque con granularidad gruesa, y es más compatible con un equipo de experiencia mixta. La Microservices Architecture solo sería preferible si la agilidad de despliegue y la autonomía de los equipos fueran la restricción principal y la consistencia pudiera relajarse a consistencia eventual entre contextos. El CTO debe aceptar que consistencia absoluta y despliegue independiente total son objetivos en tensión: cuanto más se distribuye la transacción, más se paga en coordinación, latencia, disponibilidad y complejidad operativa. Priorizar la consistencia significa elegir fronteras transaccionales amplias, una fuente de verdad compartida y *releases* coordinadas; esa es la renuncia consciente que hace posible la corrección y la auditabilidad del sistema financiero.

6. El Legado de SOA y su Influencia en Arquitecturas Modernas

La pregunta analiza cómo la Microservices Architecture aborda la reutilización de una manera fundamentalmente distinta a la Service-Oriented Architecture (SOA), separando la reutilización de lógica de negocio de la reutilización operativa, y en qué escenarios sigue teniendo sentido la orquestación explícita. La respuesta se centra en la variante de SOA con motor de orquestación, que es la que concentra las críticas, sin perder de vista que SOA también admite coreografía (Richards & Ford, 2020, cap. 16).

6.1. La reutilización en SOA y el giro de los microservicios

Primero es necesario analizar Service-Oriented Architecture (SOA). SOA se diseñó con el objetivo primordial de lograr la reutilización de software a escala empresarial (*enterprise-level reuse*) para evitar reescribir aplicaciones continuamente . Para ello, definió la capa de **Enterprise Services** (servicios atómicos compartidos) estructurados bajo un modelo de datos canónico único para toda la organización . El inconveniente de priorizar la reutilización en SOA fue el acoplamiento masivo (*coupling*): cualquier cambio en el servicio canónico compartido de cliente provocaba un efecto dominó sobre múltiples aplicaciones consumidoras, exigiendo pruebas integradas y despliegues coordinados de alto riesgo.(Richards & Ford, 2020, cap. 16).

Por su lado Microservices Architecture invierte completamente esta prioridad guiándose por el concepto Contexto Acotado (Bounded Context) de Domain-Driven Design (DDD). Dado que el objetivo principal de microservicios es lograr el mayor nivel de desacoplamiento físico y autonomía, se rige bajo la regla fundamental de **favorecer la duplicación sobre la reutilización** (*duplication over reuse*) .(Richards & Ford, 2020, cap. 17)

6.2. Reutilización de lógica de negocio frente a reutilización operativa

Reutilización de lógica de negocio. En Miroservices Architecture, en lugar de invocar por red a un servicio remoto canónico compartido para validar o consultar un cliente, lo que sucede es que cada microservicio define autónomamente las entidades y reglas de negocio necesarias para su propio contexto acotado. Si dos o más microservicios (por ejemplo, Facturación y Envíos) necesitan validar o consultar datos de clientes, prefieren duplicar la lógica o definir estructuras de datos locales e independientes. Esto evita crear dependencias de red, llamadas síncronas acoplantes y esquemas de bases de datos compartidos .(Richards & Ford, 2020, cap. 17).

Reutilización operativa. En cambio, la arquitectura de microservicios separa explícitamente la lógica de dominio de las capacidades operativas. Mientras que la lógica de negocio no se comparte para evitar el acoplamiento, las preocupaciones técnicas transversales sí se benefician de la estandarización y la reutilización (Richards & Ford, 2020, cap. 17). Para no contaminar el código de negocio ni obligar a cada equipo a reimplementar herramientas de infraestructura, se utiliza el **patrón Sidecar**: las tareas operativas —monitoreo, *logging*, métricas, *circuit breakers*— se encapsulan en un componente aislado que se ejecuta junto al microservicio. Los sidecars se conectan a través de un plano de control que constituye una **Service Mesh**, lo que permite a los equipos de infraestructura actualizar y gestionar las capacidades operativas globalmente sin modificar ni acoplar el código de negocio.

6.3. Escenarios que aún justifican la orquestación

Aunque la coreografía basada en eventos asíncronos es el patrón por defecto en microservicios por su alto desacoplamiento, existen escenarios en los que un arquitecto debe considerar la orquestación explícita, ya sea mediante un mediador local en microservicios o un orquestador de sagas:

1. **Visibilidad centralizada y control de flujos complejos.** Cuando el proceso de negocio implica múltiples pasos dependientes, condicionales o paralelos, y se requiere conocer el estado global de la transacción sin dispersar la lógica del flujo en múltiples manejadores de eventos (Richards & Ford, 2020, cap. 16).

2. **Gestión avanzada de errores y recuperabilidad.** En la coreografía pura, si un servicio intermedio falla, el resto del sistema suele continuar procesando a ciegas. La orquestación explícita permite registrar el estado del flujo en un almacén persistente, lo que faculta al sistema para pausar, aplicar reintentos o reanudar el flujo exactamente desde el punto de falla (Ford et al., 2021, cap. 11).

3. **Coordinación de transacciones distribuidas con el patrón Saga.** Cuando una operación abarca múltiples microservicios con bases de datos independientes y es necesario coordinar transacciones compensatorias —acciones de hacer y deshacer— si algún paso falla más adelante (Ford et al., 2021, caps. 11 y 12).

6.4. Ejemplo: procesamiento de un pedido

Un flujo de compra en comercio electrónico es el ejemplo clásico de un proceso complejo que se beneficia de la orquestación explícita mediante un **mediador local de pedidos** (*Saga Orchestrator* o *Order Mediator*):

- **Paso 1 (inicio):** el cliente envía la solicitud de compra (`PlaceOrder`). El mediador intercepta el evento e inicia el seguimiento del estado del flujo.
- **Paso 2 (reserva de inventario):** el orquestador envía a `InventoryService` el comando de reserva de los artículos seleccionados.
- **Paso 3 (procesamiento del pago):** tras confirmar la reserva, el orquestador invoca a `PaymentService` para efectuar el cobro. Si el pago falla —por ejemplo, tarjeta rechazada—, detecta el error e inicia una transacción compensatoria, notificando a `InventoryService` para revertir la reserva y liberar los productos.
- **Paso 4 (empaquetado y despacho):** si el pago se procesa con éxito, el orquestador ordena a `WarehouseService` empaquetar la orden y a `ShippingService` generar la guía de envío.
- **Paso 5 (notificación y cierre):** finalmente, invoca a `NotificationService` para enviar el correo de confirmación y marca la transacción como completada de forma segura.

Si este proceso se estructurara con coreografía pura, cada microservicio tendría que conocer la lógica de compensación de los demás, convirtiendo al primer servicio en un mediador disfrazado y complejo (*Front Controller*). La orquestación explícita mediante un mediador local consolida la gestión del flujo en un único punto sin acoplar los microservicios individuales ni depender de un bus de servicios empresarial central (Ford et al., 2021, caps. 11 y 12).

6.5. Síntesis

El legado de SOA no es el bus empresarial ni el motor único que gobierna toda la organización. Lo que sobrevive es la idea de que los procesos complejos necesitan una coordinación explícita, con estado, compensaciones y visibilidad, y la idea de que la reutilización es valiosa cuando se aplica en el nivel correcto. Los microservicios reinterpretan ambas ideas: la coordinación se vuelve local y acotada, y la reutilización se separa en dos planos. La lógica de negocio se reutiliza dentro del contexto delimitado o se duplica de forma consciente; la infraestructura operativa se reutiliza en la plataforma. Esa separación es la diferencia fundamental con SOA y la razón por la que los microservicios pueden conservar las ventajas de la reutilización sin heredar el acoplamiento del modelo centralizado.

En consecuencia, la orquestación sigue siendo válida cuando el coste de no tener una visión global del flujo supera el coste del acoplamiento que introduce el coordinador. Debe reservarse para procesos largos, compensables, auditables o con intervención humana; para eventos independientes y flujos simples, la coreografía y la autonomía de los servicios siguen siendo la opción más coherente con el estilo.

Referencias

Ford, N., Richards, M., Sadalage, P., & Dehghani, Z. (2021). *Software Architecture: The Hard Parts*. O'Reilly Media.

Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.

Declaración de Uso de la Inteligencia Artificial

Para la elaboración de este seminario, el equipo utilizó la herramienta DeepSeek V4.1 Flash con el propósito de generar el código LaTeX necesario para integrar las respuestas del proyecto y, después de la integración, revisar la correctitud de las referencias bibliográficas. El integrante Adrián fue quien la utilizó para integrar las respuestas en LaTeX. Asimismo, la integrante Laura Alonso Rivero utilizó Mermaid Live Editor, versión 12.2.0, junto con DeepSeek-V4.1-Flash-1017, para elaborar los gráficos en Mermaid de las preguntas 1 y 2. Todo el contenido generado fue revisado, verificado y adaptado por el equipo, quien asume la responsabilidad final del documento.